Motion Data Logger

v36 combined: how every feature works

Add-on version	1.9.9 (file Motion_Data_Logger_v36_combined.py); API version 1.3.0
Needs	Blender 3.6.0 or newer
Where	3D Viewport > Sidebar (N) > Motion Log
What it does	Logs the location and rotation of every vertex of any number of objects (default every 0.0021 s), plus a whole-object centre and rotation, shows the result in a scrollable log with a wireframe icon above each vertex column, exports it, saves it in the .blend and exposes it to other add-ons.

About the animations

Each feature page shows two frames from an animated GIF, and the 14 GIFs are delivered next to this PDF (a PDF cannot play animation). The GIFs are **schematic illustrations** drawn from the add-on's code to show the logic; they are not screen recordings of Blender, and numbers in them are illustrative unless stated.

#	Feature	Animation file
1	Tracked Objects	`01_tracked_objects.gif`
2	Sampling Settings: Store Data, Interval, Range	`02_sampling_modes.gif`
3	Log Data and Clear Data	`03_log_data_recording.gif`
4	Large-Recording Warning	`04_size_warning.gif`
5	Vertex Numbering Order	`05_vertex_order.gif`
6	Object Location (Object View)	`06_object_location.gif`
7	Rotation: Object and Per Vertex	`07_rotation.gif`
8	Display Rows: Frame Rate and Per Frame / Per Shutter	`08_display_rows.gif`
9	The Log Display: Views, Columns, Scrolling, Pop-up	`09_log_display_scroll.gif`
10	Wireframe Icons above Each Vertex	`10_wireframe_icons.gif`
11	Export Log to Text	`11_export_text.gif`
12	Save Log in .blend / Load Log from .blend	`12_save_load_blend.gif`
13	Lifecycle and Preferences	`13_lifecycle_preferences.gif`
14	Public API for Other Add-ons	`14_public_api.gif`

The panel at a glance

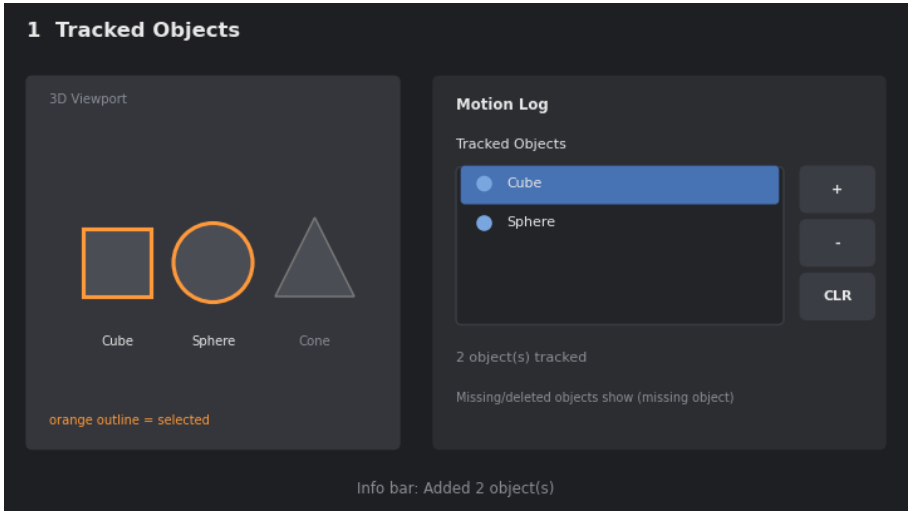
Top to bottom, as the controls appear in **Sidebar > Motion Log**:

Control	What it does	Section
Tracked Objects list, + − trash	Choose the objects to log	1
Sample Interval (s)	Time between samples (Per Shutter only)	2
Vertex Stride	Every Nth vertex for the whole-object summary	2
Use Scene Frame Range / Start / End	Which frames are recorded	2
Large recording note	Appears when the estimate is above about 1 GB	4
Log Data / Clear Data	Start recording (ESC cancels) / erase the log	3
Store Data: Per Shutter | Per Frame	How samples are stored	2
Info lines (mode, samples, time)	Plain-language summary of what will be recorded	2, 4
Display Frame Rate, Display Per	Rows shown in the log and export	8
Pop Up Log, eye toggle	Show the log in a pop-up or inside the panel	9
Export Log to Text	Write the displayed log to a .txt file	11
Save Log in .blend / Load Log from .blend	Keep the log inside the project file	12
Log display (when shown)	Views, Rows / Columns Shown, scroll sliders, table with icons	9, 10
Add-on Preferences	Remove project data when the add-on is removed	13

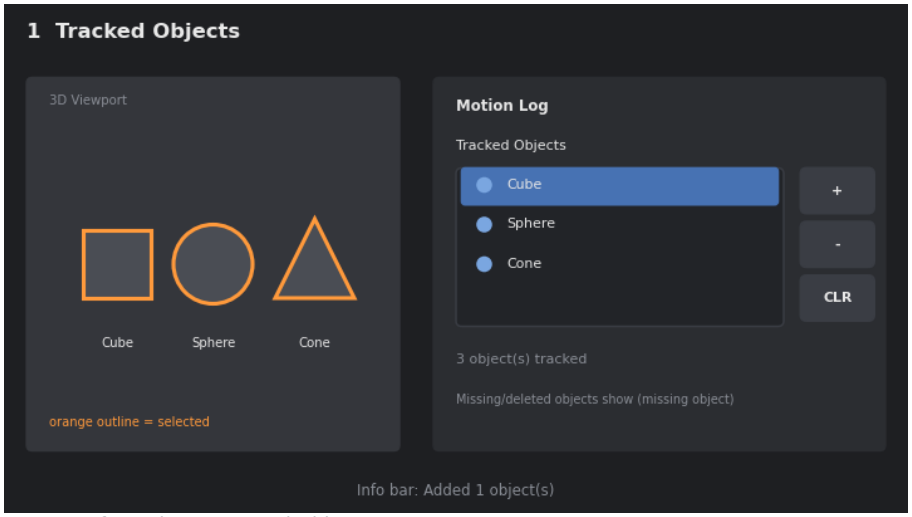
1 Tracked Objects

Choose which objects get logged. Any number of objects, any mesh.

- **+ (Add Selected Objects)** adds every selected object that is not already in the list. The info bar reports how many were new, and objects already tracked are skipped.
- **− (Remove)** deletes only the highlighted row; the highlight then moves up one row.
- **Trash (Clear List)** empties the list. None of the buttons touch the objects in your scene.
- A row whose object was deleted shows *(missing object)* and is ignored when recording.
- The highlighted row also decides which object the log display shows (section 9).
- Objects without geometry (empties, cameras, lights) can be tracked too. They log their matrix transform and have no vertex columns.



Frame 1 of 2: Two objects added with +



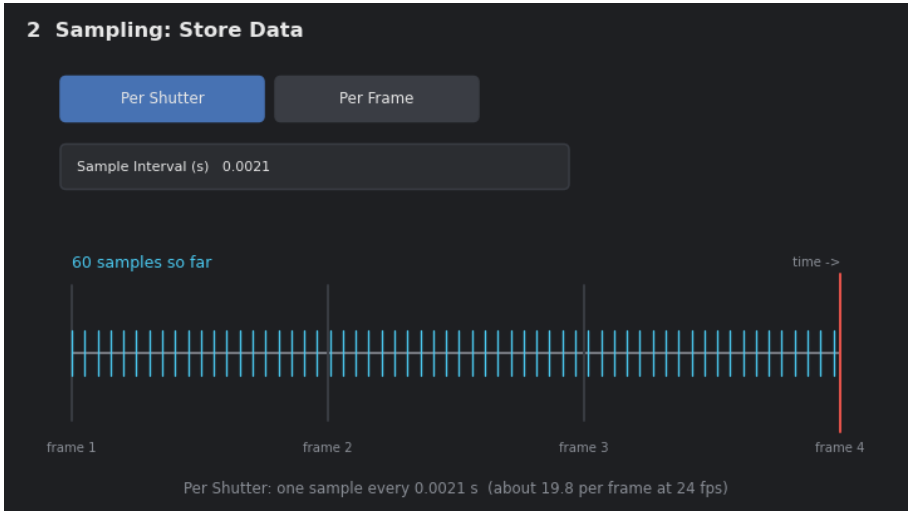
Frame 2 of 2: Sphere removed with −

Animated version: `01_tracked_objects.gif`

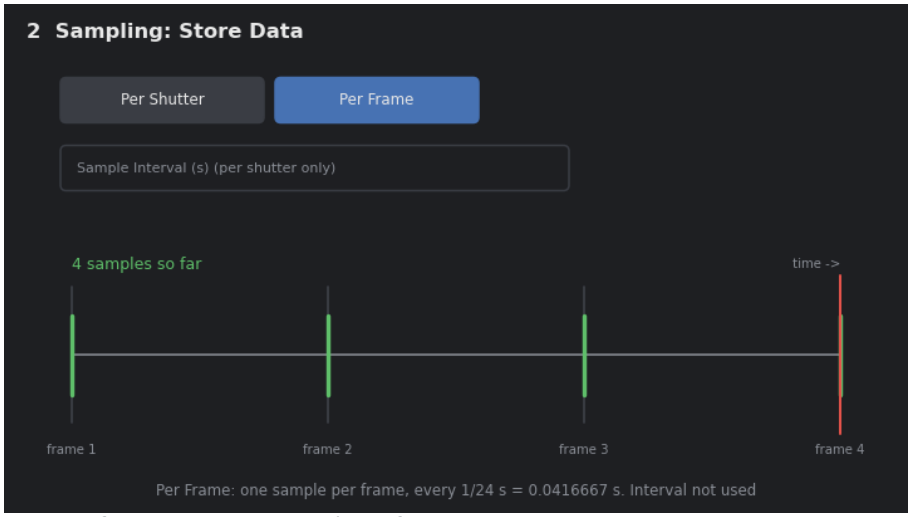
2 Sampling Settings: Store Data, Interval, Range

How often, and over which frames, the data is stored.

- **Store Data = Per Shutter**: one sample every *Sample Interval* seconds. The default 0.0021 s is the shutter-open time at 240 fps with a 180° shutter (allowed range 0.0001 to 1 s).
- **Store Data = Per Frame**: one sample per frame at the project's frame rate (interval = 1 ÷ fps). Sample Interval is greyed out and labelled *(per shutter only)*.
- Blender float properties are 32-bit, so the add-on uses the shortest decimal that round-trips the float32 (0.0021, not 0.002099999925).
- **Use Scene Frame Range** is on by default. Untick it to type your own Start and End frame.
- **Vertex Stride** (default 1) only affects the whole-object summary: every Nth vertex is used to find the centre and rotation. Individually logged vertices are never skipped.
- A line under the Log Data button states the mode (for example *One sample every 0.0021 s*) and how many samples will be logged.
- Units: location in metres (Blender units × scene unit scale), rotation as XYZ Euler degrees.



Frame 1 of 2: Per Shutter: a dense run of samples



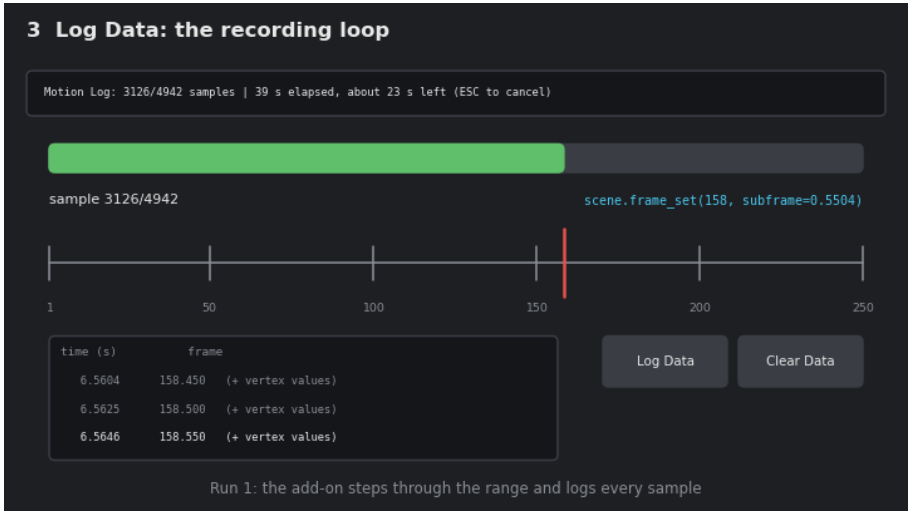
Frame 2 of 2: Per Frame: one sample per frame

Animated version: `02_sampling_modes.gif`

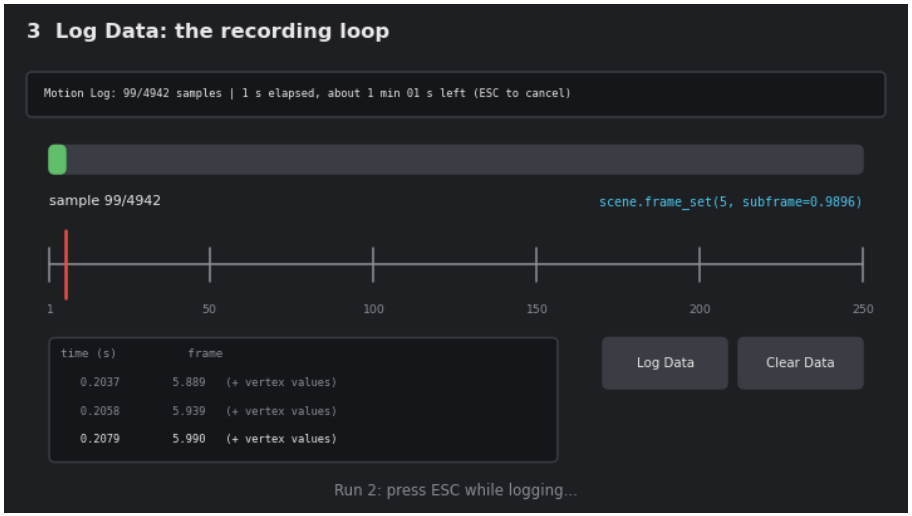
3 Log Data and Clear Data

Steps through the frame range and records every vertex of every tracked object, without freezing Blender.

- Recording is a **modal timer** (ticks every 0.01 s). Each tick samples for up to 0.08 s, so the UI, ESC and scrolling stay responsive.
- Sample count = ceil(duration ÷ interval) + 1. The last sample lands exactly on the end frame.
- For each sample the add-on calls `scene.frame_set(whole, subframe=fraction)`, reads the evaluated depsgraph and appends one row per object (object values plus every vertex).
- The area header shows samples done, time elapsed and an estimate of time left.
- **ESC** cancels. Any error (deleted object, mesh problem) also ends the recording cleanly. Samples taken so far are kept either way.
- Only one recording can run at a time, and changing scene cancels it. When it ends, your original current frame is restored.
- **Clear Data** erases the log. During a recording it stops it first, then erases.
- API subscribers receive *started, progress, finished / cancelled, cleared*.



Frame 1 of 2: Run 1: logging to completion



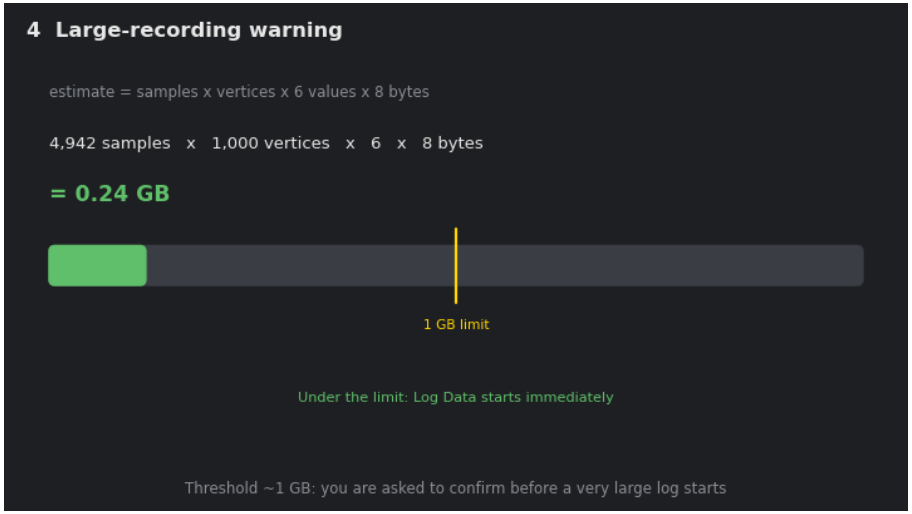
Frame 2 of 2: Run 2: ESC keeps the samples so far

Animated version: `03_log_data_recording.gif`

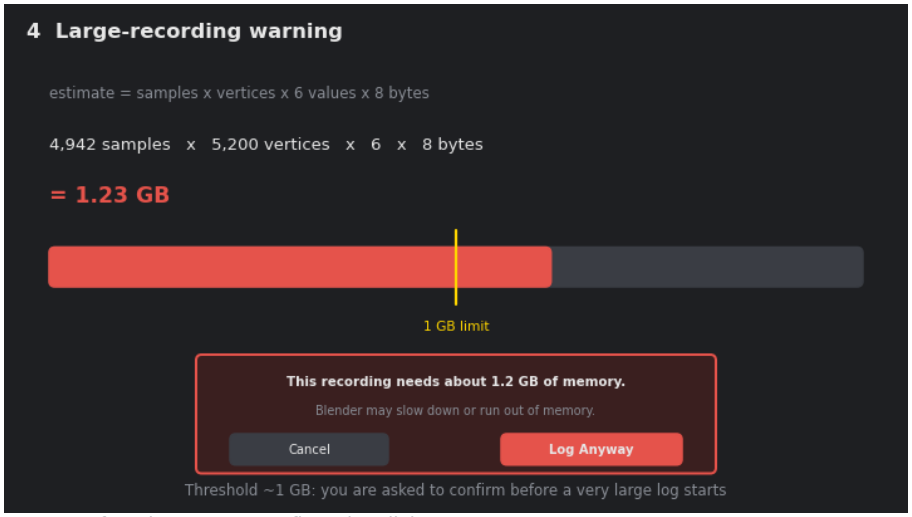
4 Large-Recording Warning

Instead of limiting how many vertices are logged, the add-on warns you when the data would get risky.

- Estimated size = samples × vertices × 6 values × 8 bytes. It uses each object's base mesh, so modifiers that add geometry can make the real size larger.
- Above about **1 GB** the panel shows a red *Large recording* note, and pressing Log Data opens a dialog with **Log Anyway**.
- The dialog suggests how to reduce the size: raise Sample Interval (Per Shutter only), shorten the frame range, or track fewer objects.
- After one recording of the same objects the add-on remembers seconds per sample, so later warnings and the panel show an expected time (for example *about 3 min 20 s*). Before that, the time left appears in the header while logging.
- There is no vertex limit: every vertex of every tracked object is logged.



Frame 1 of 2: Below the limit: starts immediately



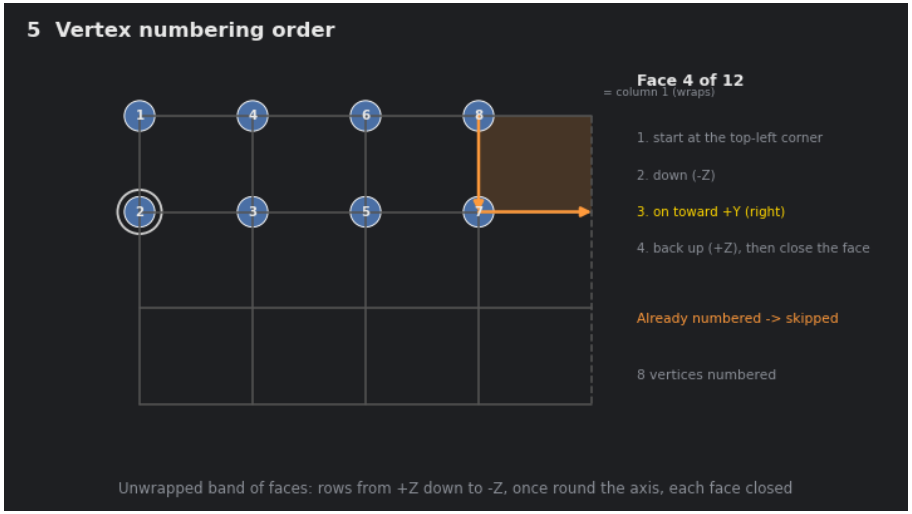
Frame 2 of 2: Above 1 GB: confirmation dialog

Animated version: `04_size_warning.gif`

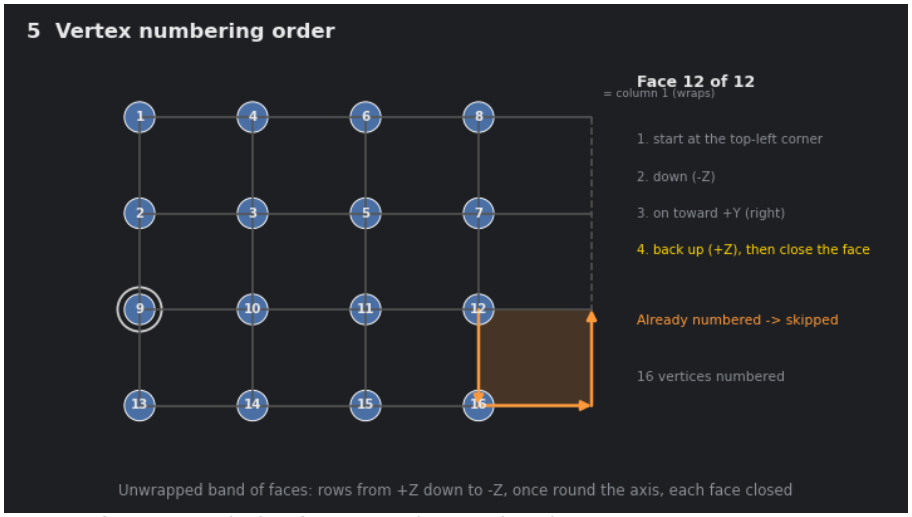
5 Vertex Numbering Order

Vertices are numbered once, from the first sample, and keep that order for the whole recording.

- **Vertex 1** is the highest-Z vertex. Ties: highest X, then closest to Y = 0, then lowest Y.
- Faces are grouped into **rows** of equal centre height and taken from +Z down to −Z. Inside a row they go once round the vertical axis, from the +X side toward +Y.
- Each face is walked: top corner → down (−Z) → on toward +Y → back up (+Z) → back to the start, which closes the face (triangle, quad or any n-gon). A perfectly flat face starts at its highest-X corner and goes counter-clockwise seen from above.
- A vertex gets its number the first time a face reaches it. Later faces only add their new corners.
- Vertices that belong to no face come last, ordered by position. A mesh with no faces is ordered by position alone.
- The animation unwraps a ring of faces into a flat grid so the path is easy to follow.



Frame 1 of 2: Face 4: starts at top-left, walks the corners



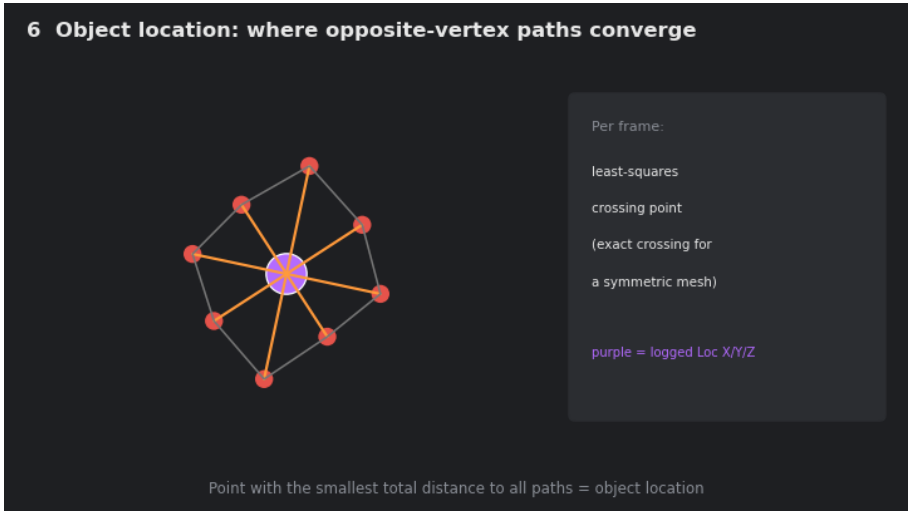
Frame 2 of 2: Face 12: the last face, 16 vertices numbered

Animated version: `05_vertex_order.gif`

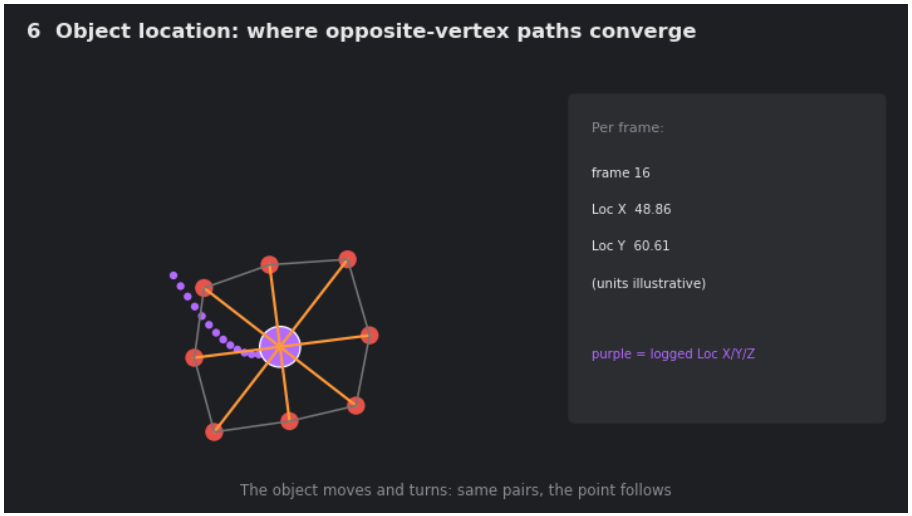
6 Object Location (Object View)

The whole-object position is found from its vertices, not from the object origin.

- Every vertex is paired with the vertex lying most directly on the opposite side of the centre. The pairs are found **once**, at the first sample (start from the middle of the bounding box, 5 refinement passes) and kept for the whole recording.
- The logged location is the point with the **smallest total squared distance** to all those straight paths. For a symmetric mesh (UV sphere, cube) every path crosses at one point and that is the answer. For any other mesh it is the closest point to all of them.
- Zero-length paths are ignored. A nearly flat set of paths is steadied by a tiny pull toward the average midpoint.
- Meshes with more than 4,000 vertices use an evenly spread subset when searching for pairs, to keep it fast.
- If no pairs can be formed, the average of the vertices is used.
- Result is in metres (× scene unit scale). Vertex Stride applies here.



Frame 1 of 2: Opposite-vertex paths and their crossing point



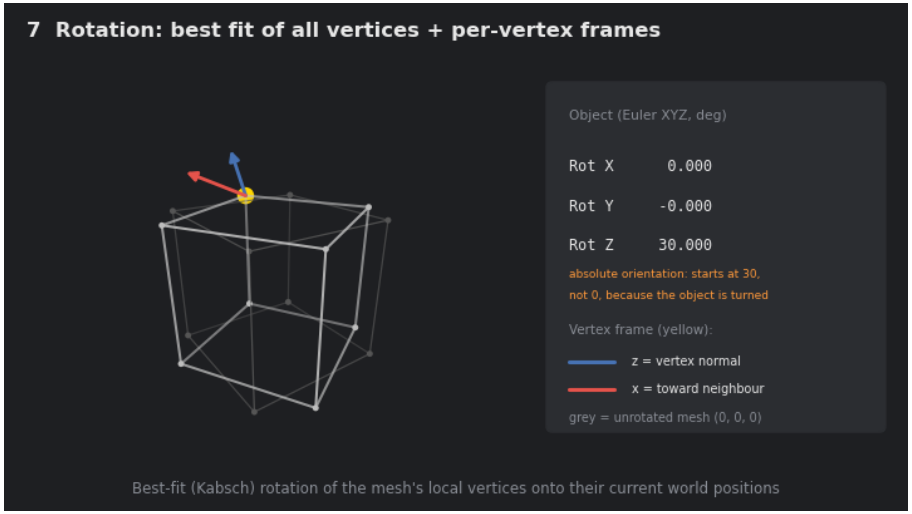
Frame 2 of 2: The point follows the moving, turning object

Animated version: `06_object_location.gif`

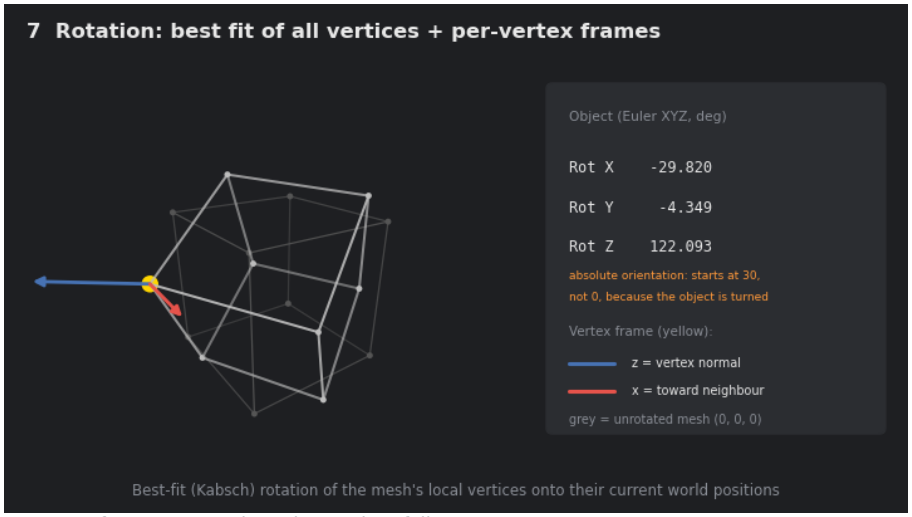
7 Rotation: Object and Per Vertex

Rotations are logged as absolute XYZ Euler degrees, kept continuous from sample to sample.

- **Whole object:** the best-fit (Kabsch, via SVD) rotation that maps the mesh's *local* vertices (as they were at the first sample) onto their current *world* positions. That is the object's absolute orientation, not a change since the first sample: an object already turned 30° when recording starts logs 30° in the first row. Objects without geometry use their matrix instead.
- **Per vertex:** each vertex gets a local frame: z = the vertex normal, x = the direction to a connected neighbour (made perpendicular to z). The frame at the first sample is the rest frame.
- A vertex's logged rotation is the object's world rotation × (its current frame × the inverse of its rest frame). For a rigid object every vertex therefore reports the object's rotation; a vertex that turns differently from the mesh (bending, deformation) reports the difference.
- Of the two equivalent Euler solutions, the one closest to the previous sample is used, and angles are unwrapped by whole turns, so values do not jump. Gimbal lock is handled.
- **Vertex location** is its world position in metres. If a vertex disappears during a recording its values are NaN and shown as “-”.



Frame 1 of 2: Starts turned: Rot Z reads 30°, not 0 (absolute)



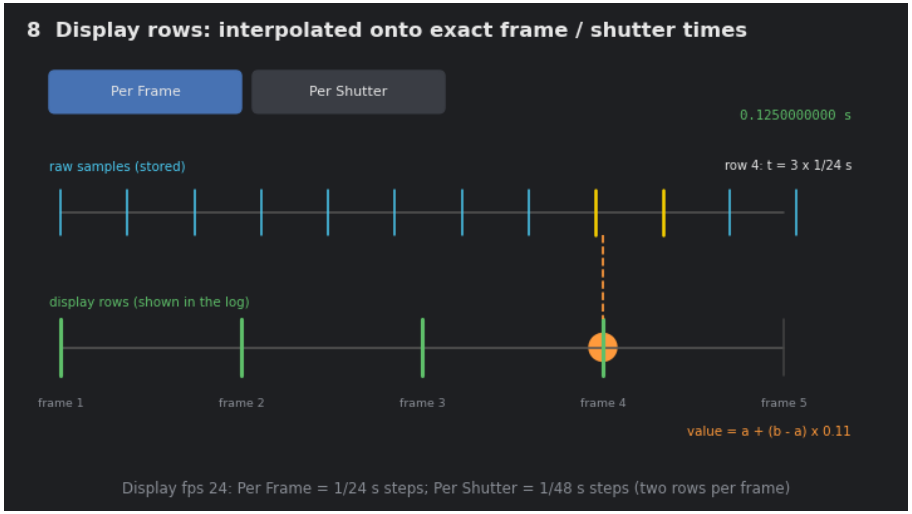
Frame 2 of 2: Later sample: Euler readout follows

Animated version: `07_rotation.gif`

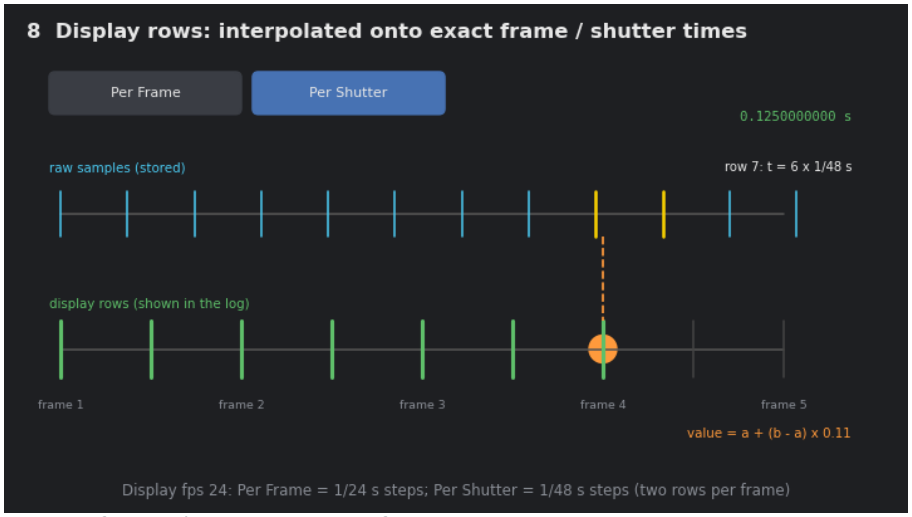
8 Display Rows: Frame Rate and Per Frame / Per Shutter

The log is stored at the recording interval but displayed at a frame rate you choose.

- **Display Frame Rate:** 12, 24, 25, 30, 48, 50, 60, 100, 120 or 240 fps (default 24).
- **Display Per = Per Frame:** one row every 1 ÷ fps. **Per Shutter:** one row every 1 ÷ (2 × fps), a 180° shutter, so two rows per frame.
- Row *k* sits exactly at *k* × *step* seconds. Its values are linearly interpolated between the two stored samples around that moment, so the Time column is exact and, when the display fps matches the scene's, so is the Frame column.
- The last row is the final sample if the recorded range ends on a row.
- Rows are computed on demand, so a big log is never copied.
- These options are independent of Store Data: they change what you see and export, not what was recorded.



Frame 1 of 2: Per Frame: rows on the frame lines



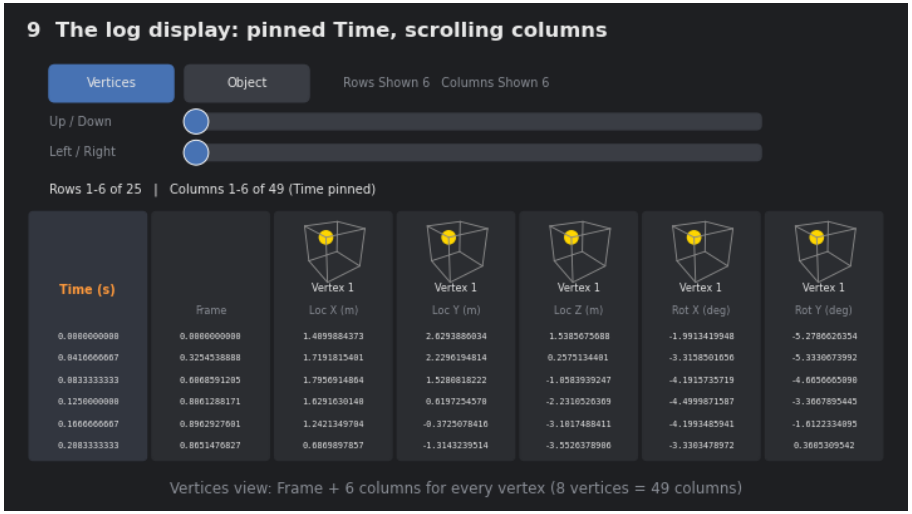
Frame 2 of 2: Per Shutter: two rows per frame

Animated version: `08_display_rows.gif`

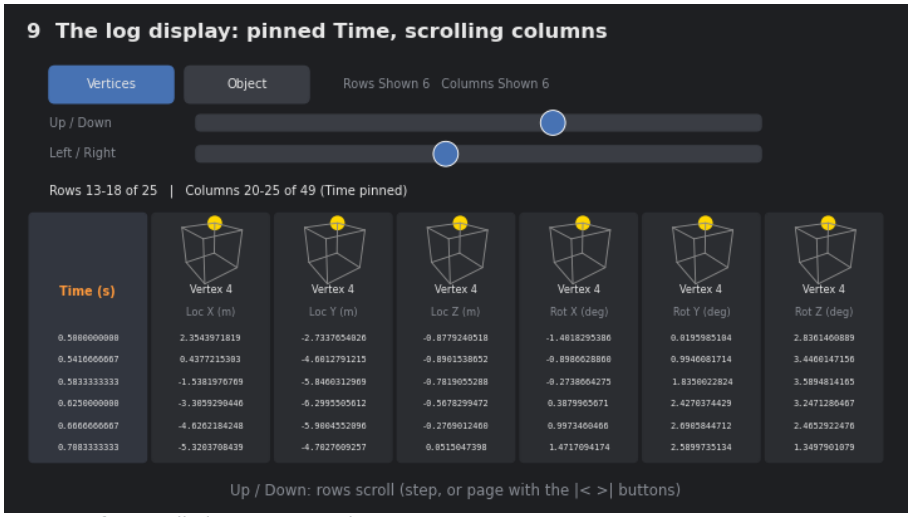
9 The Log Display: Views, Columns, Scrolling, Pop-up

A table with a pinned Time column that you scroll with the add-on's own controls.

- The eye toggle shows or hides the log inside the panel. **Pop Up Log** opens the same display in a 1100 px pop-up (move the mouse away to close it).
- **Log View = Vertices:** Frame plus six columns for every vertex. **Object:** Frame, Loc X/Y/Z, Rot X/Y/Z.
- **Rows Shown** (default 20) and **Columns Shown** (default 6) set the window size. The Time column is always pinned.
- **Up / Down** and **Left / Right** each have a slider plus step and page buttons. A status line reads *Rows 13-18 of 25 | Columns 20-25 of 49 (Time pinned)*.
- Values show 10 decimals. A cell wider than its column is stacked onto extra lines (split at the decimal point); a missing vertex shows “-”.
- A summary line gives the object name, raw sample count and storage mode, interval, and the row mode. The object shown is the highlighted tracked one, or the first logged.



Frame 1 of 2: Vertices view with the pinned Time column



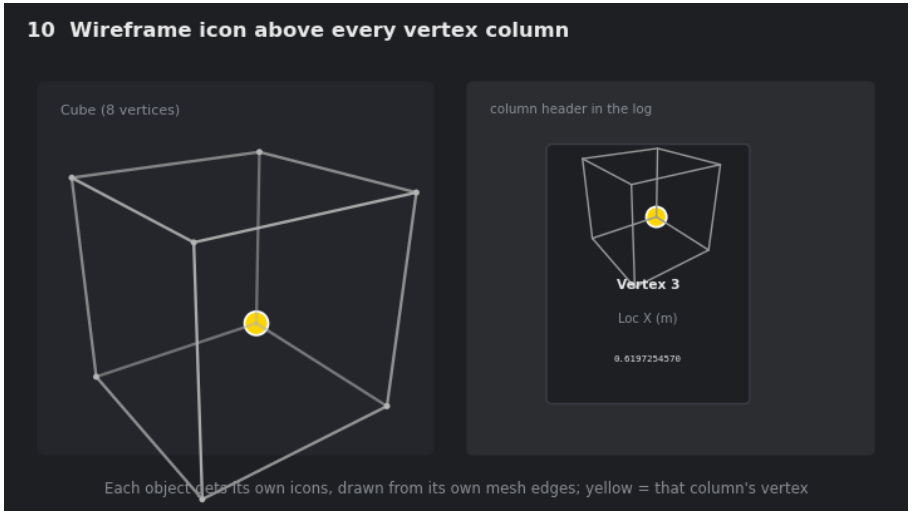
Frame 2 of 2: Scrolled to Vertex 4 columns, rows 13-18

Animated version: `09_log_display_scroll.gif`

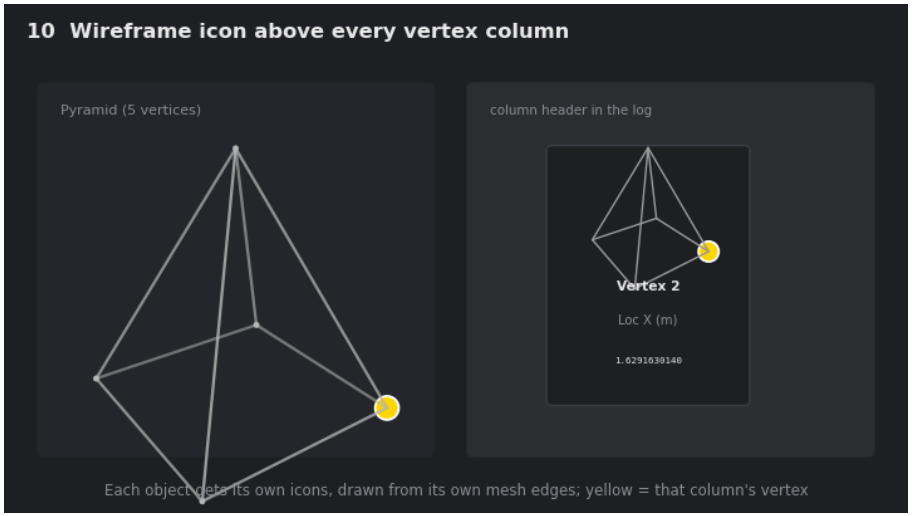
10 Wireframe Icons above Each Vertex

Each vertex column is headed by a small wireframe of its own object with that vertex marked.

- The icon draws the object's own mesh edges in grey and marks the column's vertex in **yellow** (with a thin white ring so it reads on any theme).
- Every tracked object has its own set, so a cube and an icosphere show different shapes and different vertex counts.
- Icons are drawn with numpy, saved as PNGs in a temporary folder and shown as Blender previews. They are created only when a column is first displayed, so huge meshes cost nothing up front.
- Meshes with more than 2,500 edges are thinned for the icon; vertex dots are drawn for objects up to 256 vertices.
- Icons are erased with the data and removed when the add-on is disabled.



Frame 1 of 2: Cube: vertex 3 of 8 marked



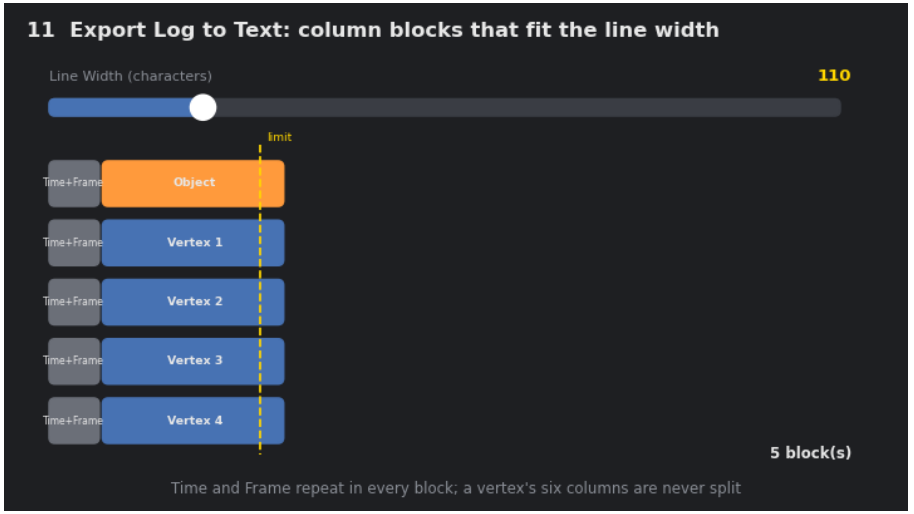
Frame 2 of 2: Pyramid: its own 5-vertex icons

Animated version: `10_wireframe_icons.gif`

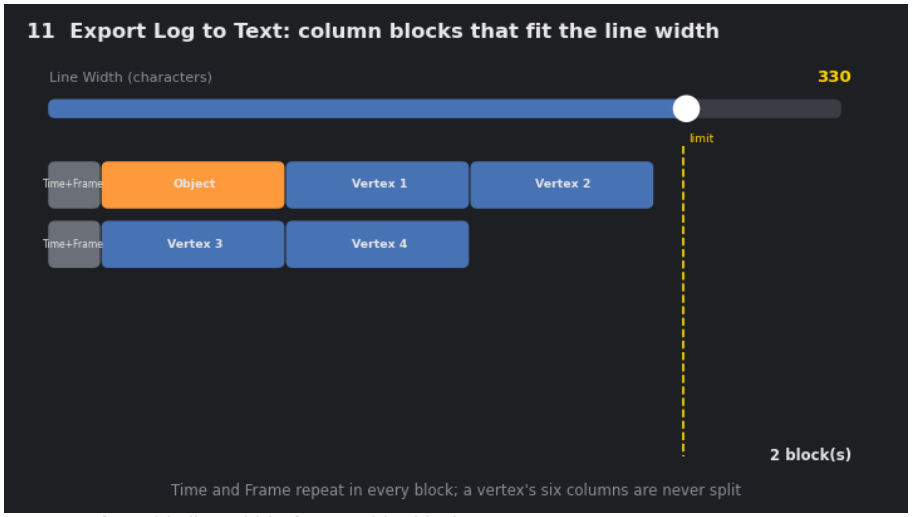
11 Export Log to Text

Writes the displayed log to a plain .txt file, laid out like the panel.

- **Export Log to Text** opens a file browser. The default name is like `MotionLog_24fps_frame.txt`, placed beside the .blend.
- One table per object, titled with the object's name and a summary line. Rows follow the chosen Display Frame Rate and Display Per.
- **Line Width** (default 110 characters, 40 to 2000): a table wider than this is split into **blocks of columns** stacked under each other. Time and Frame repeat in every block, and a vertex's six columns are never split. A single group wider than the limit still gets a block of its own.
- **Only the Highlighted Object** exports just the object the log display is showing.
- Output is UTF-8 text; a write error is reported instead of failing silently.



Frame 1 of 2: Narrow line width: many small blocks



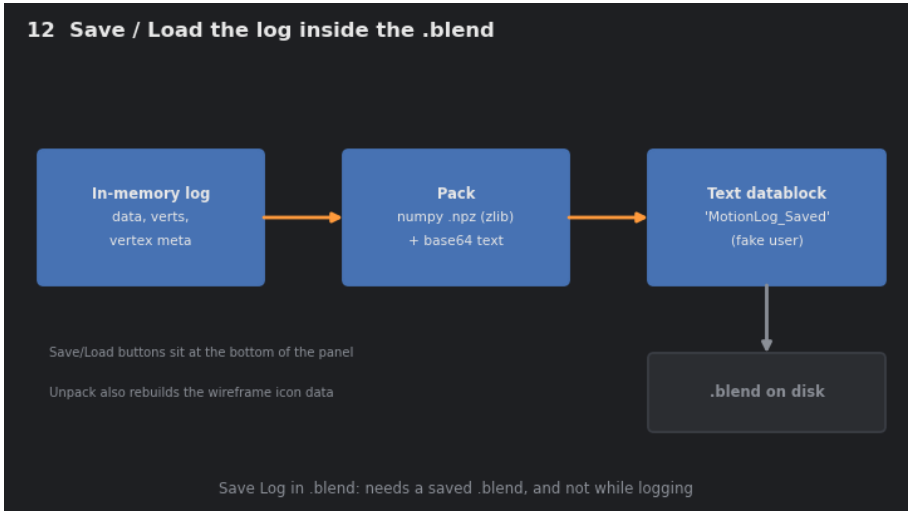
Frame 2 of 2: Wide line width: fewer, wider blocks

Animated version: `11_export_text.gif`

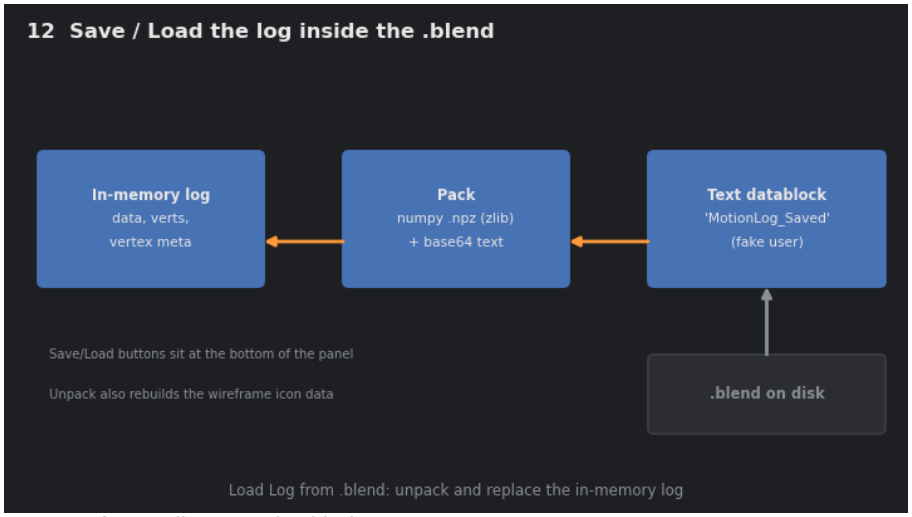
12 Save Log in .blend / Load Log from .blend

Keep the full-resolution log inside your project file.

- **Save Log in .blend** packs the log into compressed numpy data encoded as base64 text and stores it in a Text datablock named `MotionLog_Saved`, with a fake user so Blender keeps it.
- It needs a .blend already saved to disk, is blocked while logging, and reports an error if nothing is logged. Save the .blend afterwards to keep it.
- The save holds the raw samples (not just the display rows), per-vertex data, wireframe data, interval, store mode, fps and start frame.
- **Load Log from .blend** replaces the current log with the stored one (blocked while recording). Older saves without vertex data still load.
- Opening the file later restores the log automatically (section 13).



Frame 1 of 2: Saving: memory → pack → Text datablock



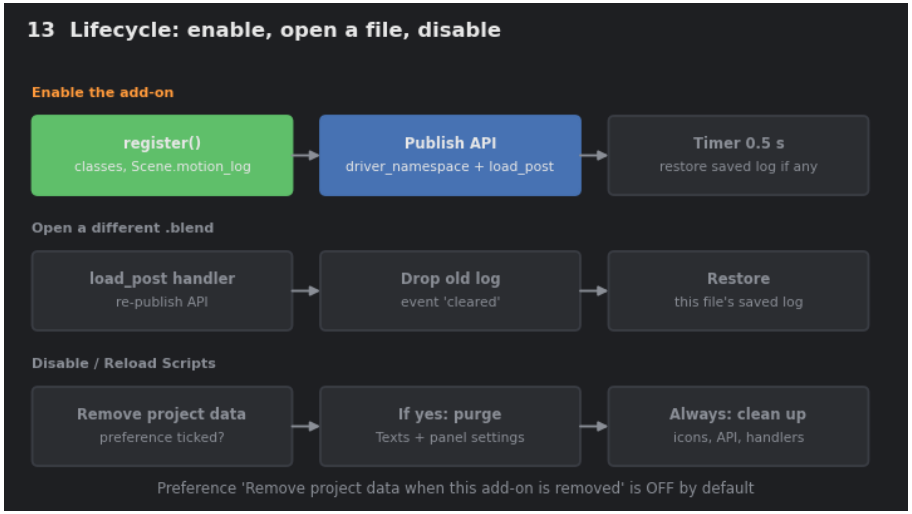
Frame 2 of 2: Loading: Text datablock → memory

Animated version: `12_save_load_blend.gif`

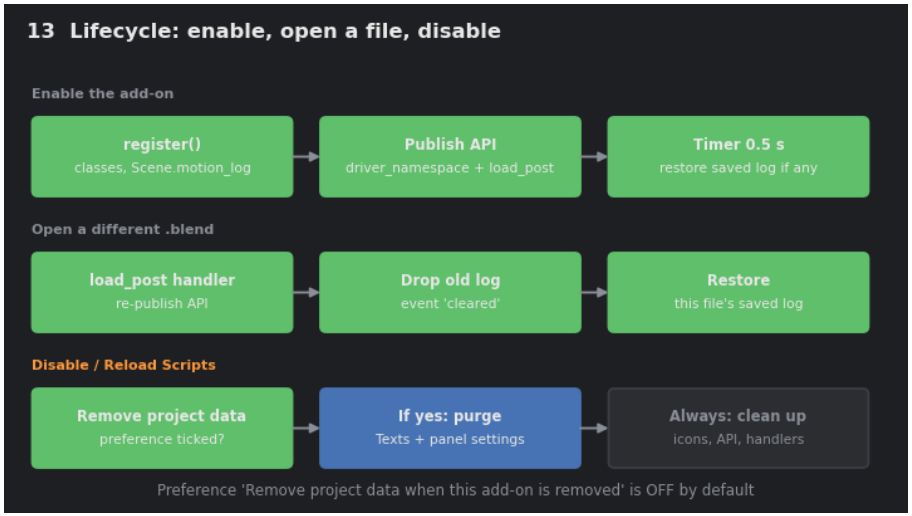
13 Lifecycle and Preferences

What the add-on does when it is enabled, when a file is opened and when it is disabled.

- **Enable:** classes and the *Scene.motion_log* settings are registered, the API is published, a file-load handler is installed, and a 0.5 s timer restores a saved log (bpy.data cannot be used inside register()).
- **Open a file:** the API is re-published, the previous file's log is dropped (event *cleared*), and the log saved in the new file is restored, so the panel never shows another file's data.
- **Preference “Remove project data when this add-on is removed”** (off by default): when ticked, disabling or Reload Scripts also deletes the saved-log Text, old MotionLog_*.csv Texts and the panel settings. Tick it *before* uninstalling.
- Disabling always stops a running recording, frees the icons and removes the API and handlers.



Frame 1 of 2: Enable and file-open steps completed



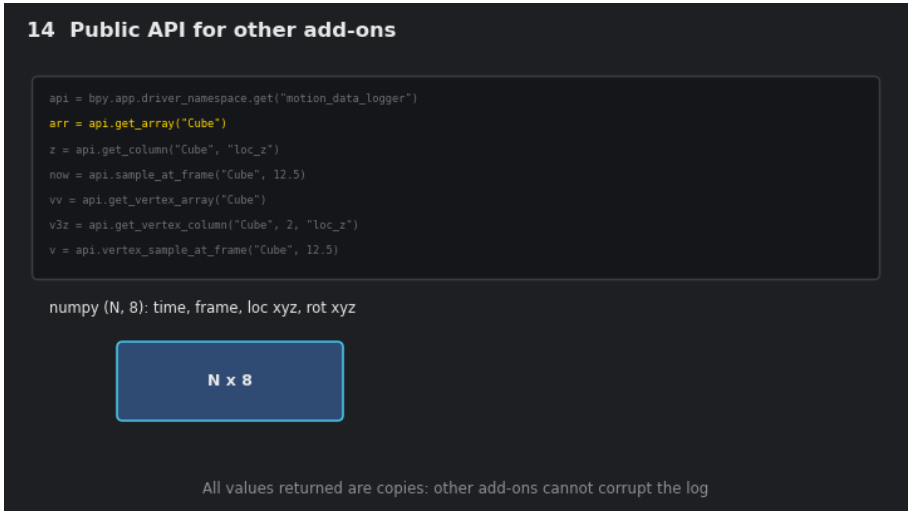
Frame 2 of 2: Disable: the project-data preference decides

Animated version: `13_lifecycle_preferences.gif`

14 Public API for Other Add-ons

While the add-on is enabled, other add-ons can read the log without importing this file.

- Get the API with `bpy.app.driver_namespace.get("motion_data_logger")`. It is re-published after every file load.
- Whole-object data: *get_array* returns an (N, 8) array: time, frame, loc xyz, rot xyz. Per-vertex data: *get_vertex_array* returns (N, K, 6).
- *sample_at_frame*, *sample_at_time* and the vertex versions return interpolated values for any (fractional) frame or time, clamped to the recording.
- *subscribe(callback)* receives *started, progress, finished, cancelled, cleared, loaded*. A failing callback never breaks recording.
- Everything returned is a copy, so other add-ons cannot corrupt the log. Full method list on the next page.



Frame 1 of 2: Reading whole-object data



Frame 2 of 2: Reading per-vertex data

Animated version: `14_public_api.gif`

API reference (section 14)

Units: time in seconds, frame in scene frames, loc in metres, rot in XYZ Euler degrees. Whole-object fields: `time, frame, loc_x, loc_y, loc_z, rot_x, rot_y, rot_z`. Vertex fields: `loc_x … rot_z`. Vertex index 0 means vertex 1.

Call or property	Returns
`interval, store_per, fps, start_frame, is_recording`	Seconds between samples; "SHUTTER" or "FRAME"; scene fps and start frame of the last recording; whether one is running
`object_names(), has_data(name=None), sample_count(name)`	Logged object names; whether data exists; samples for one object
`get_array(name), get_all()`	numpy (N, 8) for one object, or a dict of them
`get_column(name, field), get_sample(name, index)`	One field as a 1-D array; one raw sample as a dict (negative index counts from the end)
`sample_at_time(name, t), sample_at_frame(name, frame)`	Interpolated dict, clamped to the recording
`get_vertex_count(name)`	K, the vertices logged for that object
`get_vertex_array(name)`	numpy (N, K, 6): sample, vertex, loc xyz rot xyz
`get_vertex_column(name, vertex, field)`	One field of one vertex as a 1-D array
`vertex_sample_at_time(name, t), vertex_sample_at_frame(name, frame)`	{vertex_index: {field: value}}, interpolated
`subscribe(cb), unsubscribe(cb)`	cb(event, info) with started, progress, finished, cancelled, cleared, loaded

Example

```
import bpy
api = bpy.app.driver_namespace.get("motion_data_logger")
if api is not None and api.has_data():
    arr  = api.get_array("Cube")                # (N, 8)
    locz = api.get_column("Cube", "loc_z")
    now  = api.sample_at_frame("Cube", 12.5)    # now["loc_x"]
    vv   = api.get_vertex_array("Cube")         # (N, K, 6)
    v3z  = api.get_vertex_column("Cube", 2, "loc_z")  # vertex 3
```

Quick reference: settings and defaults

Setting	Default	Notes
Sample Interval (s)	0.0021	0.0001 to 1. Used only by Per Shutter
Store Data	Per Shutter	Per Frame = one sample per frame at the project fps
Vertex Stride	1	Whole-object summary only; vertices are never skipped
Use Scene Frame Range	On	Off enables Start (1) and End (250)
Display Frame Rate	24 fps	12, 24, 25, 30, 48, 50, 60, 100, 120, 240
Display Per	Per Frame	Per Shutter = two rows per frame
Log View	Vertices	Or Object (centre + best-fit rotation)
Rows Shown / Columns Shown	20 / 6	Time column always pinned
Show Log Here (eye)	Off	Pop Up Log shows the same display
Export Line Width	110 characters	40 to 2000; Only the Highlighted Object is off
Remove project data on uninstall	Off	Add-on Preferences; tick before uninstalling
Large-recording warning	above ~1 GB	Samples × vertices × 6 × 8 bytes
Saved log Text name	MotionLog_Saved	Inside the .blend, fake user